

Steam: A deconstruction of a virtual video-game store

A Database Foundations Project

1st Juan Pablo Borja Espitia
Universidad Francisco José de Caldas
System engineering
Bogotá D.C, Colombia
20202020091

Abstract—This paper deconstructs Steam’s database model, exploring its architecture, design, and interaction with backend systems. Steam, developed by Valve Corporation, is one of the most prominent platforms for video game distribution. This project analyzes the structure behind Steam’s database and examines how it manages the platform’s vast game library and user interactions. Additionally, as part of this project, a custom backend API was developed using Python to interface with a database, demonstrating fundamental principles of database architecture and integration. The implementation provides insights into data retrieval, storage, and communication between the API and the database, offering a practical perspective on database design.

Index Terms—Database, Model, Steam, Architecture, Back-end API, Data Retrieval

I. INTRODUCTION

Steam is a video game online distributor developed by Valve Corporation back in 2003, was originally designed as a digital distribution platform for Valve’s own games like Half life 2 which over time more games got added to the platform, and one day Valve would allow users to upload their own games as well, which evolved into one of the most prominent online marketplaces for video game distribution ever made, hosting thousands of titles from various developers worldwide.

II. BUSINESS MODEL

Steam operates on a commission-based revenue model, in which the platform retains a percentage of all game sales made through its marketplace. This model has been instrumental in shaping the digital distribution industry, with platforms such as Amazon also adopting similar strategies to monetize digital content effectively.

In addition to commission-based sales, Steam was a pioneer in the free-to-play model during its early years. By offering free-to-play games, the platform successfully attracted a larger user base, increasing user engagement and retention. This strategy not only contributed to Steam’s dominance in the gaming industry but also influenced the widespread adoption of free-to-play monetization mechanisms, such as in-game purchases and microtransactions, which remain central to many modern gaming platforms [1][2].

III. METHOD AND MATERIALS

Given Steam’s extensive game library and millions of active users, its database architecture must efficiently manage large-scale data operations, including user interactions, transactions, and content delivery. This project aims to analyze the structure of the Steam database, focusing on its core components. Furthermore, as part of this project, a back-end API was developed using Python to interact with a database system. This implementation serves as a practical demonstration of database management principles, applying things like data queries and a back-end system integration on a web page or software. By bridging theoretical database analysis with real-world application, this research contributes to a deeper understanding of database design and its role in large-scale digital platforms.

A. Entity Model

The steam application usually revolves about 2 main entities, games and users, which basically are the core of the entire platform, taking into account that these are the next entities involved:

- User: This entity is basically the client and how the client should be handled in the database, an user can have a list of games, a list of friends, and the personal information such as the username, phone, country, e-mail and most importantly, its password.
- Game: This entity is the main product of Steam, everything within Steam revolves around games, and this entity may have a name, a genre, a developer, and a cost, in a less detailed way.
- Genre: This is pretty much a list of genres that share a reference to the game’s genres.

Then most of the diagram is pretty much done, in this case, following the same logic for steam’s library feature and list of friends, 2 more entities had to be added called "Friend" and "Library" which basically acts as reference of relation between entities, like friends being 2 references of 2 different users, and library being the reference of a game id and an user

id, this technical decision has been done to reduce complexity on the database development.

B. ERM Model

The project follows a structured methodology that involves ten steps in creating an ERM, which visualizes the interrelations between the entities in the database. These steps include defining components, identifying entities and attributes, establishing relationships, and producing diagrams to illustrate the data structure.

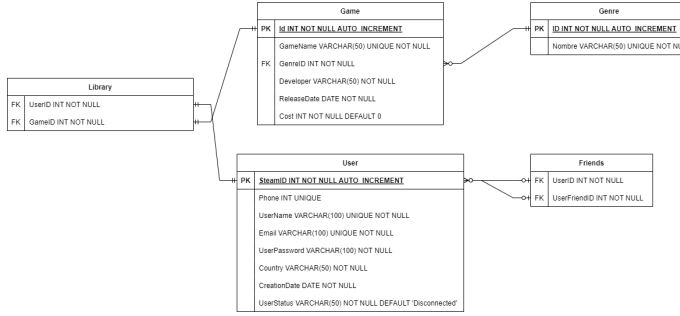


Fig. 1. ERD of the App

C. SQL Database Implementation

Once the ERM is done, the next step would be create the SQL script that sets everything on a SQL database within a server, SQL, which stands for Structured Query Language, is a programming language that's used to retrieve, update, delete, and otherwise manipulate data in relational databases. As the name suggests, MySQL is a SQL-based relational database designed to store and manage structured data [4]. In this case, for the development of the application, the next script is added to initialize the database:

```
CREATE DATABASE IF NOT EXISTS steamproject;
USE steamproject;
```

Once the backend builds up, it creates the database automatically if it wasn't already created, to make sure it doesn't overwrite.

```
CREATE TABLE IF NOT EXISTS User (
    SteamID INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    UserName VARCHAR(100) UNIQUE NOT NULL,
    Phone INT UNIQUE,
    Email VARCHAR(100) UNIQUE NOT NULL,
    UserPassword VARCHAR(100) NOT NULL,
    Country VARCHAR(50),
    CreationDate DATE NOT NULL,
    UserStatus VARCHAR(50) NOT NULL DEFAULT 'Disconnected'
);
CREATE TABLE IF NOT EXISTS User (
    SteamID INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    UserName VARCHAR(100) UNIQUE NOT NULL,
    Phone INT UNIQUE,
    Email VARCHAR(100) UNIQUE NOT NULL,
    UserPassword VARCHAR(100) NOT NULL,
    Country VARCHAR(50),
    CreationDate DATE NOT NULL,
    UserStatus VARCHAR(50) NOT NULL DEFAULT 'Disconnected'
);
CREATE TABLE IF NOT EXISTS Friends (
    UserID INT NOT NULL,
    UserFriendID INT NOT NULL,
```

```
PRIMARY KEY (UserID, UserFriendID),
FOREIGN KEY (UserID)
REFERENCES User(SteamID) ON DELETE CASCADE,
FOREIGN KEY (UserFriendID)
REFERENCES User(SteamID) ON DELETE CASCADE
);
CREATE TABLE IF NOT EXISTS Genre (
    ID INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    Nombre VARCHAR(50) UNIQUE NOT NULL
);
CREATE TABLE IF NOT EXISTS Game (
    Id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    GameName VARCHAR(50) NOT NULL,
    GenreID INT NOT NULL,
    Developer VARCHAR(50) NOT NULL,
    ReleaseDate DATE NOT NULL,
    Cost INT NOT NULL DEFAULT 0,
    FOREIGN KEY (GenreID) REFERENCES Genre(ID)
);
CREATE TABLE IF NOT EXISTS SteamLibrary (
    UserID INT NOT NULL,
    GameID INT NOT NULL,
    PRIMARY KEY (UserID, GameID),
    FOREIGN KEY (UserID)
REFERENCES User(SteamID) ON DELETE CASCADE,
FOREIGN KEY (GameID)
REFERENCES Game(Id) ON DELETE CASCADE
);
```

In this snippet, the tables are being created, in the case of SteamLibrary and Friends, the table only has 2 primary keys that reference each other, this solves a lot of complexity when doing certain queries into the database.

```
INSERT INTO Genre (Nombre) VALUES ('Action');
INSERT INTO Genre (Nombre) VALUES ('Adventure');
INSERT INTO Genre (Nombre) VALUES ('Casual');
INSERT INTO Genre (Nombre) VALUES ('Indie');
INSERT INTO Genre (Nombre) VALUES ('Multiplayer');
INSERT INTO Genre (Nombre) VALUES ('Racing');
INSERT INTO Genre (Nombre) VALUES ('RPG');
```

It is automatically added some preset genres in order to use within the database, as stated before, it'll create a primary key and a genre's name to the table, which then the game table can reference anytime in its queries.

D. API RESTFUL Implementation

To demonstrate the practical application of database modeling, this project implements a relational database integrated with a RESTful API using FastAPI which is a web framework for building APIs with Python based on standard Python type hints.[3] The system is designed to handle data interactions efficiently while ensuring scalability and maintainability. The following key technical decisions were made:

- 1) Implementation of the database: A relational database (in this case, this project will use MySQL engine) was selected due to its structured schema, ACID compliance, and robust support for data integrity. The ERM serves as the foundation for designing tables, relationships, and constraints.
- 2) RESTful API Design: The FastAPI framework was chosen to create a lightweight, asynchronous, and high-

performance API. FastAPI provides built-in data validation, automatic OpenAPI documentation, and seamless integration with databases.

- 3) Data Modeling: The ERM was translated into database tables with appropriate primary keys, foreign keys, and constraints, ensuring data consistency and reducing redundancy.
- 4) Scalability and maintainability: The RESTful API architecture allows for modular development, meaning that new features can be integrated without disrupting the existing system. In addition, asynchronous request handling in FastAPI improves efficiency by reducing response times for database queries.
- 5) Database Management and Visualization: This project also uses DBeaver, a multi-platform database management tool, and it was used for querying, managing, and visualizing the database schema, ensuring efficient debugging and monitoring of data relationships.

E. API RESTFUL Design

The api uses 4 essential commands that can apply perfectly to the CRUD operations of SQL syntax:

- GET: This service acts as a 'SELECT' command to SQL, the API has these GET services:
 - /user/get_by_id/{id_} This service gets a user from the database based on its id.
 - /user/get_all This service gets all the users from the database.
 - /user/get_by_name/{name} This service gets a user from the database based on its name.
 - /user/get_by_email/{email} This service gets a user from the database based on its email.
 - /user/get_by_country/{country} This service gets a user from the database based on its country.
 - /user/get_by_phone/{phone} This service gets a user from the database based on its phone.
 - /game/get_by_id/{id_} This service gets a game from the database based on its id.
 - /game/get_all This service gets all the games from the database.
 - /game/get_by_genre This service gets all the games from the database based on the genre.
 - /game/get_by_developer This service gets all the games from the database based on the developer.
 - /game/get_by_name This service gets all the games from the database based on the game name.
 - /game/get_genres This service gets all the genres from the database.
 - /friend/get_all This service gets all the friends references data.
 - /friend/get_by_user_id/{user_id} This service gets all the users info from the repository based on the user id in the friends table.

- /friend/is_friend/ This service checks if the user is already friends with the friend id.
- /library/get_all This service gets all the library references data.
- /library/get_games_by_user_id/ This service gets all the games info from the repository based on the user id in the library table.
- /library/has_game/ This service checks if the user has already this game.
- /library/get_user_by_games_id/{game_id} This service gets all the users who has the game in the library table.

- POST: This service acts as a 'INSERT' command to SQL, the API has these POST services:

- /user/create This service creates a new user in the database.
- /game/create This service creates a new game in the database.
- /friend/create This service creates a new friend reference in the database.
- /library/create This service creates a new friend reference in the database.

- PUT: This service acts as a 'UPDATE' command to SQL, the API has these PUT services:

- /user/update/{id_} This service updates the user data in the database based on its id.
- /game/update/{id_} This service updates the game data in the database based on its id.

- DELETE: This service acts as a 'DELETE' command to SQL, the API has these DELETE services:

- /user/delete/{id_} This service deletes a user from the database based on its id.
- /game/delete/{id_} This service deletes a game from the database based on its id.
- /friend/delete/{user_id}/{friend_id} This service deletes a friend reference from the database based on the user id and friend id.
- /library/delete/{user_id}/{game_id} This service deletes a friend reference from the database based on the user id and friend id.

For the back-end, this project is composed by multiple modules within the app:

- connections: This module basically handles the connection to the MySQL database from python using MySQL.connector, which basically enables Python programs to access MySQL databases.

```
def connect(self):
    """This method connects to the MySQL database."""
    try:
        self.connection = mysql.connector.connect(
            database=self._dbname,
            user=self._duser,
            password=self._dpass,
            host=self._dhost,
            port=self._dport,
            auth_plugin='mysql_native_password',
        )
    except Error as e:
        print(f"MySQLConnectionError:{e}")
```

- **crudOperations:** This module contains every method the api can do via messages, each class contains the respective methods of every entity, following a format for each one so the responses get the same format for most of the queries. All the classes must contain a database connection variable, which references to the databaseConnection object in order to connect with the database every time the api receives a method.

```
def __init__(self, db_connection: db_connection):
    self.database_connection = db_connection
    self.database_connection.connect()
```

- **DataAccessObjects:** This module contains every object format for the response body on the api, each format should be almost the same as the one in the SQL code, for example:

```
"""This module defines the user data object."""
from datetime import datetime
from pydantic import BaseModel, Field

class UserData(BaseModel):
    """This class represents an user data object"""
    steam_id: int = Field(default=None)
    username: str
    phone: int
    password: str
    email: str
    country: str
    creation_date: datetime = Field(default_factory=datetime.now)
```

- **dbinit:** This module contains every MySQL script that can be used within the program, in this case, the script that initializes the database.
- **services:** This module contains every router access for the api in order to link each crud method of the api with a directory, so any app can access through the methods using the routers, for example:

```
@router.put("/user/update/{id_}",
            response_model=bool)
def update(id_: int, data: UserData):
    try:
        crud.update(id_, data)
        return True
    except Exception as e:
        raise HTTPException(status_code=404,
                             detail=str(e)) from e
```

```
@router.delete("/user/delete/{id_}",
               response_model=bool)
def delete(id_: int):
    try:
        crud.delete(id_)
        return True
    except Exception as e:
        raise HTTPException(status_code=404,
                             detail=str(e)) from e
```

```
@router.get("/user/get_by_id/{id_}",
            response_model=Optional[UserData])
def get_by_id(id_: int):
    user = crud.get_by_id(id_)
    if user == 'No user found':
        raise HTTPException(status_code=404,
                             detail="User not found")
    return user
```

- **main.py:** At last there's the main class that executes everything related to the api, then, The docker build must

be executed in order to setup the MySQL database using the next command and the api itself:

```
docker-compose up --build
```

- **Dependencies and Libraries:** The project has a requirements.txt that contains each library and dependency needed in order to execute on any device that can run python and docker-compose.

F. DQL Operations Usage

The DQL Operations (Data Query Language) are computer languages that are used to make various queries in information systems and databases.[5] The api uses some of these DQL Operations like for example:

```
SELECT * FROM Game
WHERE Id = 1
```

This DQL Operation basically selects one or many records on the GAME Table where the Id value is "1", there's also this type of Query used in the api game/has_game/ where you can check if an user already has the game, which is useful in the steam app itself, this is the query:

```
SELECT COUNT(*) FROM SteamLibrary
WHERE (UserID = 1 AND GameID = 2)
```

This operation will select all game foreign keys that are equal to 2, and check as well if the userId on that record of SteamLibrary is equal to 1, this will select all games with gameid 2 that are basically own by the user with id as 1, and returns the count of how many where found, if it's 0, this means the user doesn't own this game or the gameid is not registered at all.

IV. RESULTS AND APPLICATION

A. Example of usage of the API

- 1) The client deploys the docker build, this is how the backend-app should behave on the logs, which shows up the process of the build being executed, take note that the mysql database will initialize first before executing the python app.

```
2025-02-15 00:50:16 INFO:
Will watch for changes in these directories: ['/SteamApp']
2025-02-15 00:50:16 INFO:
Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2025-02-15 00:50:16 INFO:
Started reloader process [1] using StatReload
2025-02-15 00:50:18 INFO:
Started server process [8]
2025-02-15 00:50:18 INFO:
Waiting for application startup.
2025-02-15 00:50:18 INFO:
Application startup complete.
```

- 2) Now the client can connect to the api using localhost:8000 as the address, and then be able to use any method that the api offers, let's say for example that the clients wants to add a new User using localhost:8000/user/create/ that will be ask for a json file to load up every information required for the user in order to create a new one succesfully, this is an example of how the body should be:

```

{
  "steam_id": 0,
  "username": "string",
  "phone": 0,
  "password": "string",
  "email": "string",
  "country": "string",
  "creation_date": "2025-02-15T06:24:06.002Z"
}

```

Note that the steamID value has to be in the body but the api won't register that info, instead it'll create an available id for the user, once it creates a new user, the response should return the new steamID of this user, now the client knows what steamID its new user has.

V. CONCLUSIONS

- It has been concluded that Steam relies heavily on games and users, meaning that the database model relationships usually aim to the users and the games.
- It was also concluded that the design of this model might be just a lite version of the actual database of steam due to it's complexity and size.
- It was concluded that the usage of RESTFUL API for web services is actually very useful and easier to implement for SQL databases because of how the similarities between both syntax actually makes sense.
- Most of the use of the Database model had to be changed in order to reduce complexity in most of the DQL operations.

REFERENCES

- [1] "Valve Corporation". Valve Corporation. [Online]. URL: <https://www.valvesoftware.com/es/>
- [2] SQLearning. Entity Relationship Model (ERM). <https://sqlearning.com/sql-server-introduction/entity-relationship-model/>, July 11th, 2022
- [3] "FastAPI". FastAPI. Accessed on february 13th, 2025. [Online]. URL: <https://fastapi.tiangolo.com>
- [4] "MySQL Explained: Your Guide to Mastering This Powerful Database". [Online]. URL: <https://www.oracle.com/mysql/what-is-mysql/>
- [5] "Data Query Language (DQL) — GATE Notes". BYJUS.. [Online]. URL: <https://byjus.com/gate/data-query-language-dql-notes/>